Evaluating Tree-Based Data Structures

Introduction

A tree represents data as nodes connected by parent–child relationships. A root begins the hierarchy, and subtrees organize related nodes. In a binary search tree (BST), each node has at most two children: smaller keys belong to the left subtree and larger keys to the right. This ordering supports an ordered key–value map (Sedgewick & Wayne, 2011).

BST Operations

Search compares a target with successive nodes and chooses one branch. Insertion follows the same path; an existing key replaces its value. Deletion must reconnect children while preserving order. The project uses the in-order successor, the next larger key. These operations cost O(h), where h is tree height (Sedgewick & Wayne, 2011).

Practical Benefits

In-order traversal retrieves sorted keys in O(n) time for n entries. The project therefore produces ordered collections without sorting each request. Linked nodes accommodate updates without shifting a contiguous sequence. Structural inspection supports hierarchical reasoning, although a search tree represents key order rather than a domain's containment rules.

Limitations

Sorted insertion creates a chain with $h = n - 1$ and linear searches. Child references and mutable invariants add storage and implementation costs; memory was not measured. Recursive implementations also face Python's finite recursion limit (Python Software Foundation, n.d.). Explicit stacks avoid that limit here, but detecting imbalance does not restore balance.

Performance Comparison

Project measurements used nine repetitions of 4,000 queries, two warmups, and identical seeded workloads. At 1,000 entries, median batch means for TreeMap/ListMap hits were 0.313/7.321 microseconds with median-first insertion and 17.305/8.531 with sorted insertion. Heights were 9 and 999. Favorable structure supported fast lookup; a chain could lose to sequential search. Single-machine batch means do not establish universal speed ratios.

Real-World Applications

Ordered collections, time-keyed event indexes, and hierarchical information benefit when ordering or parent–child navigation matters. Database indexing illustrates a related design: PostgreSQL uses balanced, multiway B-trees rather than ordinary binary trees (PostgreSQL Global Development Group, n.d.). Such production structures address constraints beyond this educational implementation.

Conclusion

Tree-based structures are most beneficial when ordered access and dynamic updates justify their complexity and suitable shape can be maintained. For small collections or unfavorable shapes, sequential storage may be preferable. Workload evidence should guide selection.

References

PostgreSQL Global Development Group. (n.d.). *B-tree indexes*. PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/btree.html

Python Software Foundation. (n.d.). *sys—System-specific parameters and functions*. Python 3.14 documentation. https://docs.python.org/3.14/library/sys.html#sys.getrecursionlimit

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.